GRIDGUARD KHARTOUM

PPO-Assisted Load Shedding and Energy Management for Developing Countries

Graduation Project Final Report

Murtada Asim Ibrahim Bakhit (18-317)
Suliman Elawad Suliman Babiker (18-310)
Abdalhay Abass Mobarak Abass (15-306)

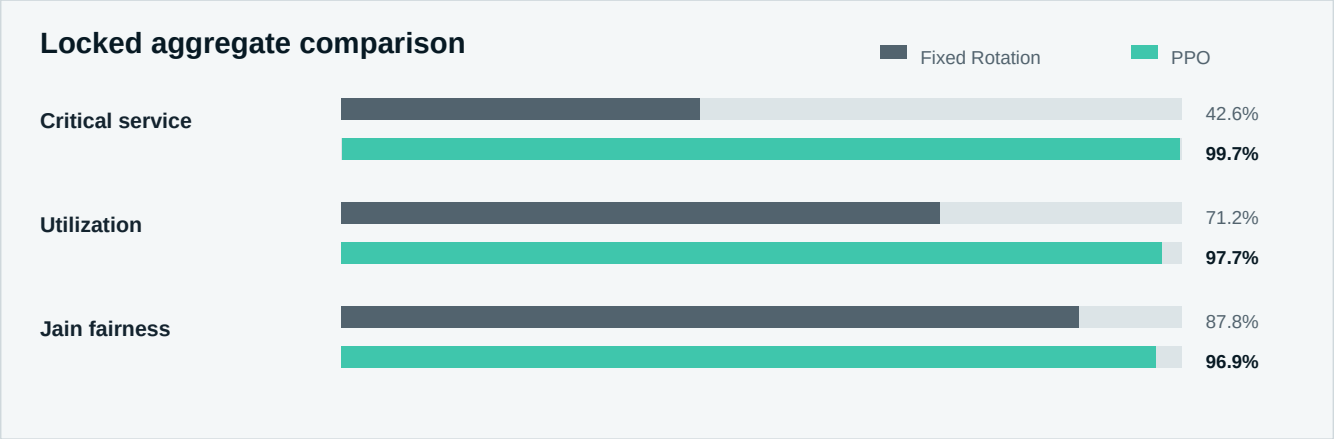
Offline research simulation | Audited synthetic evidence | GPU-first runtime

August 2026

Abstract

GridGuard Khartoum is an offline, GPU-first research simulation for proposal-aligned load shedding across seven Khartoum State localities. It compares a frozen PPO controller with Fixed Rotation on identical synthetic 24-hour shortage scenarios. The release preserves the accepted aggregate comparison and regenerates auditable paired episode evidence from the frozen PPO checkpoint and reviewed simulator. A native desktop application provides an interactive schematic map and deterministic live replay. This is decision-support research, not a deployed electrical-grid controller, and it makes no field-performance claim.

96 independent base draws x 3 shortage levels = 288 matched scenario conditions; 576 controller-scenario evaluations.



Outcome	Fixed Rotation	PPO	Paired change
Critical service	42.63%	99.72%	57.09%
Utilization	71.22%	97.65%	26.43%
Served energy	4,551.7 MWh	6,560.2 MWh	2,008.5 MWh

Contents

Abstract 2

System at a glance 4

Desktop presentation 5

Methodology 7

- Study objective and claim boundary . . . 7
- Simulation domain . . . 7
- Controller decisions and dispatch . . . 8
- Reward . . . 8
- Outcome definitions and signs . . . 9
- Matched evaluation design . . . 9
- Paired uncertainty calculation . . . 10
- Causal display-only demand projection . . . 11
- Reproducibility and provenance . . . 11
- Limitations and threats to validity . . . 12

Implementation 13

- System overview . . . 13
- Module map . . . 14
- Domain and tensor contracts . . . 14
- GPU-first execution . . . 15
- PPO artifact and inference . . . 15
- Scenario, dispatch, and metrics pipeline . . . 15
- Paired evaluation and statistics . . . 16
- Evidence provenance and fail-closed behavior . . . 17
- Display-only forecast . . . 17
- Offline schematic map . . . 17
- Desktop presentation . . . 17
- Runtime resources and packaging . . . 18
- Verification strategy . . . 18
- Deliberate implementation limits . . . 19

Results 20

- Result statement . . . 20
- Two evidence layers . . . 20
- Accepted aggregate comparison . . . 20
- Reproduced paired aggregate comparison . . . 21
- Paired differences and 95% intervals . . . 21
- Consistency across the nine evaluation cells . . . 22
- Parity with the accepted aggregate evidence . . . 22
- Interpretation against the proposal questions . . . 23
- Limitations of the result . . . 23

Conclusion 24

References 25

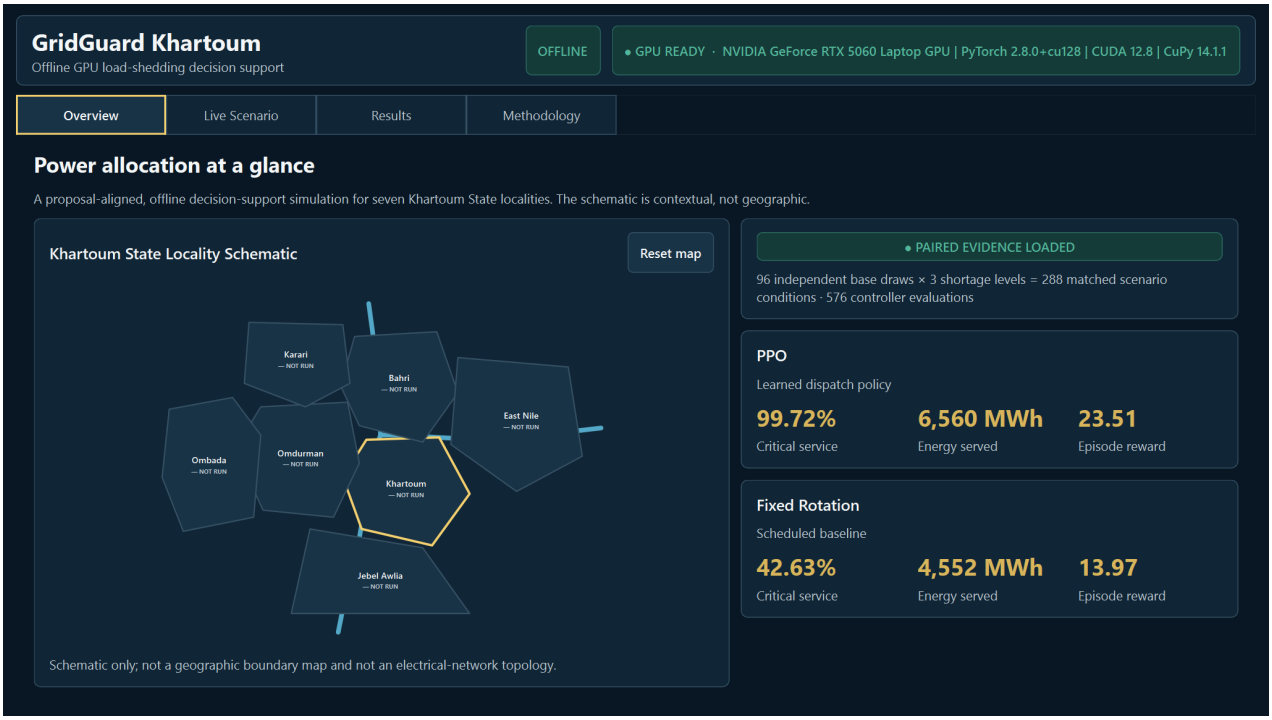
System at a glance



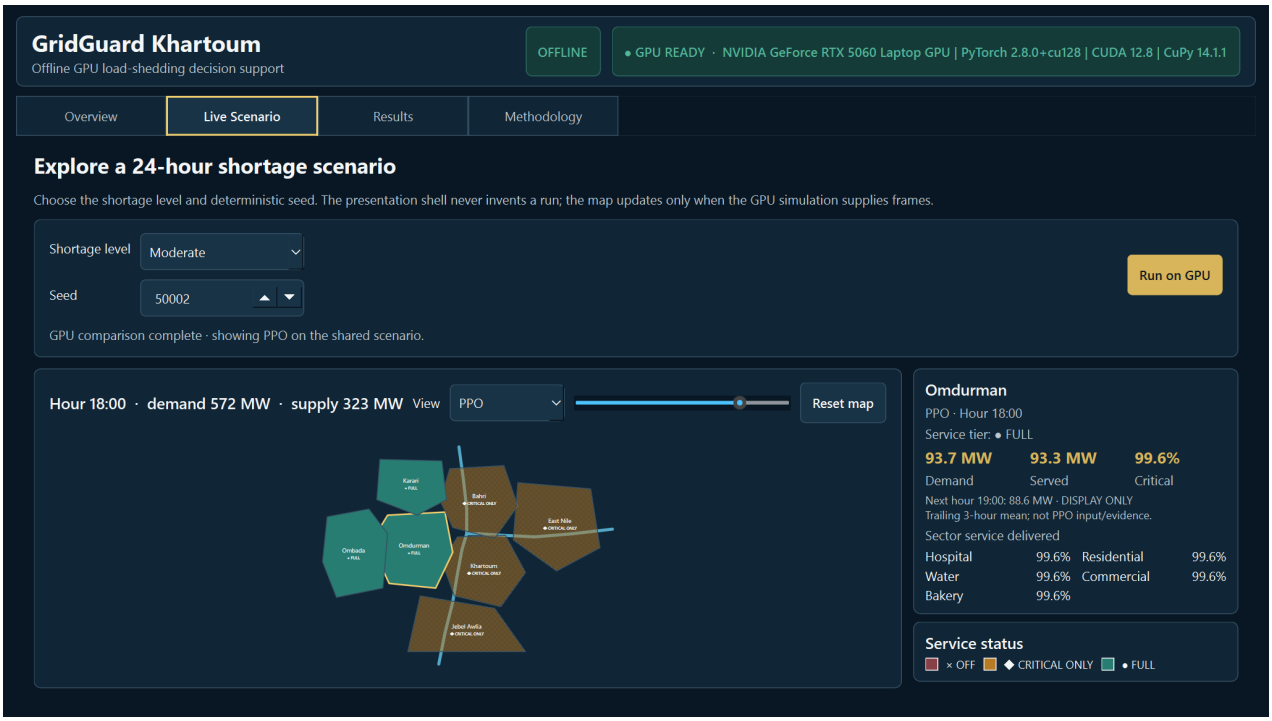
The same seeded scenario tensor is passed to both controllers. All performance-critical simulation, policy inference, metric, and bootstrap operations remain on cuda:0; final display and canonical evidence payloads cross explicitly to host memory. Startup verifies the NVIDIA runtime, evidence hashes and semantics, PPO artifact, and a CUDA smoke inference before the first presentation window appears.

Claim boundary: every numeric result in this report describes the locked synthetic simulator and protocol. It is not operational evidence from Sudan's electrical grid.

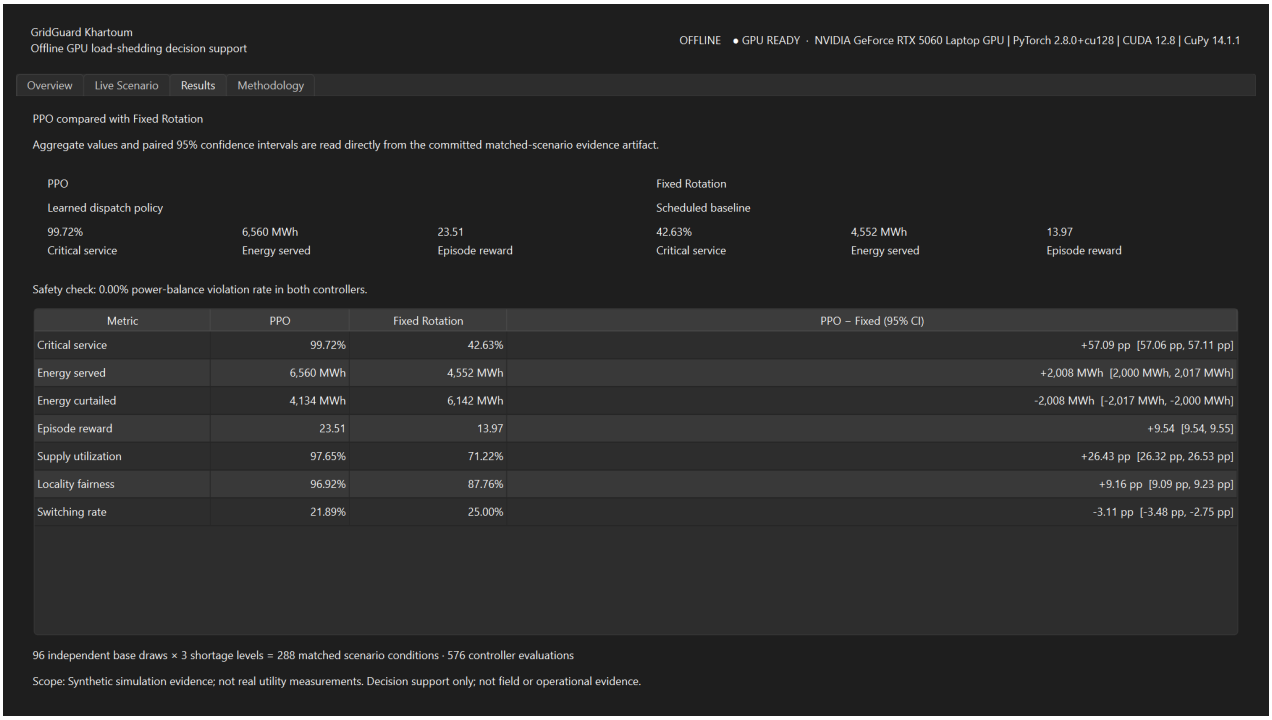
Desktop presentation



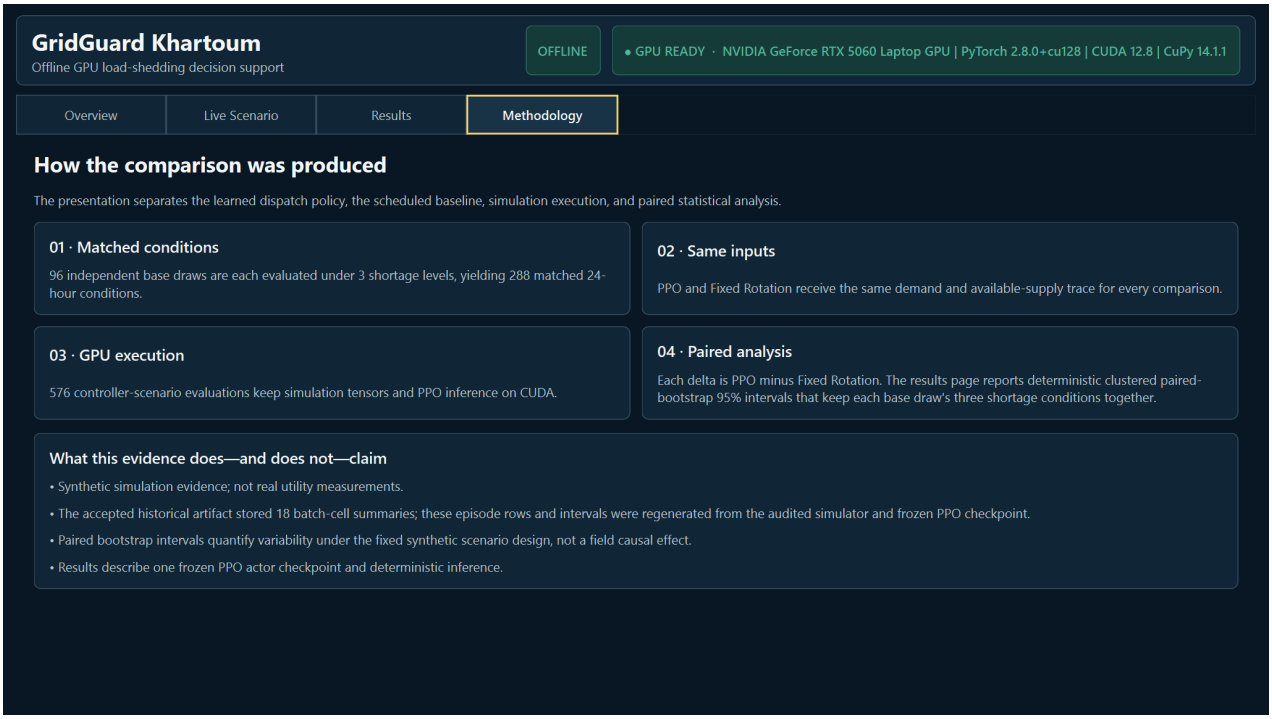
Overview - native offline desktop view captured on the presentation laptop.



Live scenario - native offline desktop view captured on the presentation laptop.



Verified results - native offline desktop view captured on the presentation laptop.



Methodology - native offline desktop view captured on the presentation laptop.

Methodology

Study objective and claim boundary

GridGuard Khartoum evaluates whether a trained PPO controller can allocate limited electricity more effectively than a traditional Fixed Rotation schedule in a simplified, software-only model of Khartoum State. The accepted proposal [1] defines this as a decision-support simulation, not a controller for a real electricity network. Accordingly, the evidence supports a comparison inside the stated synthetic scenario design only.

The confirmatory comparison is PPO versus Fixed Rotation. The proposal permits prediction, rule-based logic, optimization, and reinforcement learning where appropriate; it does not require every method to be a controller in the final comparison. The presentation includes a small causal demand projection as a separate explanatory feature. It is not an input to PPO, does not change dispatch, and is excluded from every reported performance claim.

Simulation domain

The model uses seven Khartoum State localities in a fixed tensor order: Khartoum, Omdurman, Bahri, East Nile, Jebel Awlia, Karari, and Ombada. Each locality contains five demand sectors: Hospital, Water, Bakery, Residential, and Commercial. Hospital, Water, and Bakery are the three critical sectors.

Each episode contains 24 one-hour steps. Demand and supply are expressed as MW at each step; energy totals are the sum over the one-hour steps and are therefore reported in MWh. The simulator uses one global supply pool. It does not model feeders, voltages, frequencies, transmission constraints, or power flow.

Synthetic demand

The system peak parameter is 700 MW. Locality peak shares are proportional to the locked weights below. Locked sector coefficients define the base peak allocation before distinct hourly profiles are applied, so instantaneous sector proportions can vary by hour.

Locality	Relative weight
Khartoum	1.00
Omdurman	0.95
Bahri	0.85
East Nile	0.80
Jebel Awlia	0.70
Karari	0.75
Ombada	0.90

Sector	Base peak coefficient	Profile assumption
Hospital	0.08	Nearly constant through the day
Water	0.07	Morning and evening peaks
Bakery	0.05	Early-morning and afternoon peaks
Residential	0.55	Morning and stronger evening peaks
Commercial	0.25	Midday peak

The sector curves are deterministic normalized profiles built from constant and Gaussian components. Each generated episode then receives independent uniform demand perturbations in the interval [-5%, +5%]. The perturbations are produced by a seeded PyTorch generator on the selected device.

Synthetic supply and shortage levels

Available supply is current total demand multiplied by a shortage fraction and an independent uniform perturbation in [-5%, +5%]. It is capped at 95% of demand so every episode remains a shortage case.

Shortage level	Base supply fraction	Additional shortage window
Mild	0.80	None
Moderate	0.65	Subtract 0.10 during simulation hours 17-20
Severe	0.50	Subtract 0.15 during simulation hours 16-21

The same seed reproduces the same batched demand and supply tensors in the pinned CUDA environment. The fixed evaluation seeds are 50001, 50002, and 50003.

Shortage variants are intentionally coupled. For a fixed `(seed, batch_index)` pair, the generator reuses the same demand perturbations and supply-noise draw for Mild, Moderate, and Severe; only the shortage fraction and window change. The design therefore contains 96 base draw clusters, each observed under three shortage levels, rather than 288 independent random draws.

Controller decisions and dispatch

At every hour, each controller chooses one service tier for each locality:

- Off: request no load.
- Critical Only: request Hospital, Water, and Bakery load.
- Full Service: request all five sectors.

The simulator concatenates the requested critical and non-critical loads. If total requested power exceeds available supply, one proportional scale factor is applied to every requested load. This guarantees that simulated served power does not exceed the global supply pool.

Fixed Rotation

Fixed Rotation divides the day into three-hour blocks. In each block, three consecutive localities receive Full Service and the remaining localities are Off. The starting locality advances by three positions at each block and wraps around the seven-locality order. The rule does not observe current demand, supply, critical-service need, or cumulative fairness.

PPO

PPO [2] receives a causal 32-value observation at each hour:

- four global values: sine and cosine of hour, supply divided by demand, and supply divided by the 700 MW system peak; and
- four values for each of seven localities: normalized current demand, current critical-sector share, cumulative service ratio, and previous service tier divided by two.

The inference-only actor has two 512-unit tanh layers and 2,187 output logits. The 2,187 outputs represent all `3^7` joint locality actions. Deterministic argmax inference selects one joint action, which is decoded into seven service tiers using the pinned little-endian base-3 encoding. The actor is loaded from a hash-verified, pickle-free NumPy artifact and runs on `cuda:0` in inference mode.

This release reproduces the frozen actor's architecture, weights, actions, and evaluation. It does not include optimizer state and does not claim to reproduce the original training run.

Reward

The accepted hourly reward combines critical service, utilization, fairness, switching, and oversubscription:

```
r_t = (5 C_t + 2 U_t + 2 J_t - 0.25 S_t - 3 O_t) / 9
```

Here, `C_t` is the critical-service ratio, `U_t` is utilization, `J_t` is Jain fairness [3] of cumulative locality service ratios, `S_t` is the fraction of localities that changed tier, and `O_t` is positive requested-power excess divided by demand. Switching is zero at the first hour. Episode reward is the sum of the 24 hourly rewards.

For locality `i`, let `x_i,t` be cumulative served energy divided by cumulative demand through hour `t`. The hourly fairness term is:

```
J_t = (sum_i x_i,t)^2 / (7 sum_i x_i,t^2 + 1e-8)
```

Using cumulative ratios makes fairness depend on service history rather than only the current hour.

The reward is a research design choice, not an economic valuation of electricity or service interruption. The individual physical and service metrics are therefore reported alongside reward.

Outcome definitions and signs

All ratios are unitless and remain in the interval [0, 1] unless a paired difference is being reported.

Outcome	Definition	Preferred sign for `PPO - Fixed Rotation`	Role
Episode reward	Sum of 24 accepted hourly rewards	Positive	Primary
Critical-service ratio	Critical-sector energy served / critical-sector demand	Positive	Primary
Utilization	Mean hourly served power / `min(supply, demand)`	Positive	Primary
Jain fairness	Mean hourly Jain fairness of cumulative locality service ratios	Positive	Primary
Served energy	Sum of dispatched energy	Positive	Primary
Curtailed energy	Demand minus served energy	Negative	Primary
Switching rate	Mean hourly fraction of localities changing tier; hour zero is zero	Negative	Secondary
Oversubscription	Mean positive requested-power excess / demand	Negative	Diagnostic
Power-balance violation rate	Fraction of hours served power exceeds supply by more than 0.001 MW	Non-positive	Safety
Critical outage locality-hours	Locality-hours with critical-sector service below 99%	Negative	Diagnostic
Demand and available energy	Scenario inputs summed over the episode	Not comparative	Diagnostic

Because every time step is one hour, power sums in the energy metrics have the numerical unit MWh. Critical outage locality-hours can range from 0 to 168 in one episode: 24 hours multiplied by seven localities.

Matched evaluation design

The evaluation is a deterministic, within-scenario paired comparison.

Design element	Locked value
Shortage levels	Mild, Moderate, Severe
Seeds per shortage level	50001, 50002, 50003
Cells	9
Episodes per cell	32
Independent `(seed, batch_index)` base-draw clusters	96

Design element	Locked value
Shortage variants per base draw	3
Matched scenario conditions	288
Controllers evaluated per scenario	2
Controller-scenario evaluations	576
Episode horizon	24 one-hour steps

For each shortage-level and seed cell, the generator creates one 32-episode `Scenario` batch. The exact same immutable-reference `Scenario` object is passed first to Fixed Rotation and then to PPO; the two controllers therefore receive identical demand and supply tensors. Metrics remain on the GPU until the final serialization boundary. After the one packed host transfer, the canonical artifact builder deterministically materializes episode rows and computes the small row, cell, and aggregate summaries in NumPy/Python. Every stored paired difference is defined as:

```
delta = PPO - Fixed Rotation
```

This sign convention means positive values favor PPO for higher-is-better metrics, while negative values favor PPO for lower-is-better metrics.

The estimand is the equally weighted mean paired difference across the nine shortage-level and seed cells. Equal weighting is exact here because every cell contains 32 variants. The 288 rows are valid matched controller comparisons, but their 96 base draws are the sampling clusters. The estimand does not estimate the real-world prevalence of any shortage level or seed-defined scenario family.

Paired uncertainty calculation

Confidence intervals are calculated only from the newly reproduced per-episode evidence. The accepted frozen artifact contains cell summaries rather than episode rows and therefore cannot support confidence intervals, p-values, or episode-level win rates.

For each inferential metric, the 288 paired differences are arranged as a `[3 shortage levels, 3 seeds, 32 base draws, metric]` tensor. Each of 10,000 bootstrap replicates samples 32 base-draw indices with replacement within each seed. The same sampled index is reused for that base draw's Mild, Moderate, and Severe variants, so correlated shortage variants stay together. The replicate statistic is the equally weighted mean over shortage levels, seeds, and sampled base draws. The reported 95% interval uses an explicit CuPy CUDA sort followed by linear interpolation at the 2.5th and 97.5th percentile ranks.

This is a deterministic seed-stratified paired cluster percentile bootstrap [4]. It holds the three shortage levels and three seed strata fixed; it does not resample shortage levels or seeds. This preserves the intended dependence and stratification but limits the intervals to the locked design.

The bootstrap seed is 20260815. Differences and reductions use float64. PyTorch [5] CUDA tensors are exposed to CuPy [6] through zero-copy DLPack; sampling and percentile calculation stay on `cuda:0`. Mean, lower, and upper statistics return through one stacked DLPack transfer, and the evaluator performs one final packed device-to-host transfer for canonical JSON serialization.

These intervals quantify resampling variability of the 96 base-draw clusters conditional on the nine synthetic cells. They are not p-values, do not estimate a field causal effect, and do not establish performance on a real Sudanese electricity network.

The reproduced critical-service estimand first calculates a ratio for each episode and then averages those episode ratios. The accepted cell summaries used pooled critical energy. Their maximum cell difference is `0.000001268461346626 2817`, which is below the locked `0.000002` tolerance. The new rows should therefore be understood as a source-compatible episode-level reproduction, not reconstructed historical episode data.

Causal display-only demand projection

The desktop presentation can display a one-hour-ahead demand projection. For each locality and sector, the default projection is the mean of the preceding three observed hours. A target is never included in its own input window, and changes to future observations cannot affect an earlier projection. The operation remains on the input PyTorch device.

With a three-hour window, current hours 2 through 22 display locality projections for hours 3 through 23. The first two hours have insufficient history and the final hour has no next-hour target, so the UI shows no projection for those edge cases.

This projection is intentionally simple. It demonstrates causal prediction logic for decision-support presentation, but it is not consumed by the PPO observation, simulator, reward, evaluation, or frozen claim. No forecast-accuracy claim is made.

Reproducibility and provenance

The proposal, accepted source benchmark, source checkpoint, clean inference artifact, frozen benchmark, and paired benchmark are bound by SHA-256 manifests. The key final evidence hashes are:

Artifact	SHA-256
Accepted proposal PDF	`3432e64a21ab9741bf47646fd0f4f561522ddfd81d4005deb892d38672c65e48`
Frozen aggregate benchmark	`3d720ba87a747a5780119decd7144b4051fc36fc72ab50f229f5750ad749fd15`
PPO inference artifact	`3e4a7ce44267dd1cf3bbde7e77abafb8f31fdea77ece63acda7be17fa3198ad5`
Paired per-episode benchmark	`2c3e71e48c4c7bff4fe609d42a76ca4f770f6dd717cd7d45d5e8e0d99fdf2d11`
Project source manifest	`cc06df2492512d1c4de9c73051311f3401d5ddf90eb2563f2f440e53f26194c9`

Run the release checks from the repository root on the presentation laptop:

```
uv sync --frozen --extra dev --extra docs
$sourceArchive = (Resolve-Path -LiteralPath "..\aipower_rl" -ErrorAction Stop).Path
$env:GRIDGUARD_SOURCE_REPOSITORY = $sourceArchive
$env:GRIDGUARD_SOURCE_CHECKPOINT = (
    Resolve-Path -LiteralPath (
        Join-Path $sourceArchive "artifacts\checkpoints\best.pt"
    ) -ErrorAction Stop
).Path
uv run gridguard-doctor
uv run pytest -ra
uv run gridguard-evaluate --verify-evidence
uv run ruff check .
```

The explicit archive variables turn a missing source archive or checkpoint into a test failure instead of an optional skip. Release review must confirm that pytest reports zero skipped tests. The full test command is deliberately unfiltered: release verification requires the NVIDIA GPU, the CUDA PyTorch/CuPy stack, and the trusted source archive. The CPU-safe development subset is useful feedback but is not a release certificate:

```
uv run pytest -m "not gpu and not source_archive"
```

The paired artifact can be independently regenerated to a temporary file without overwriting the committed evidence. The command refuses to target the trusted artifact and accepts the reproduction only when its bytes produce the pinned SHA-256:

```
$reproduction = Join-Path $env:TEMP "gridguard-paired-benchmark.json"
uv run gridguard-evaluate --reproduce-paired --output $reproduction
```

The required output digest is `2c3e71e48c4c7bff4fe609d42a76ca4f770f6dd717cd7d45d5e8e0d99fdf2d11`.

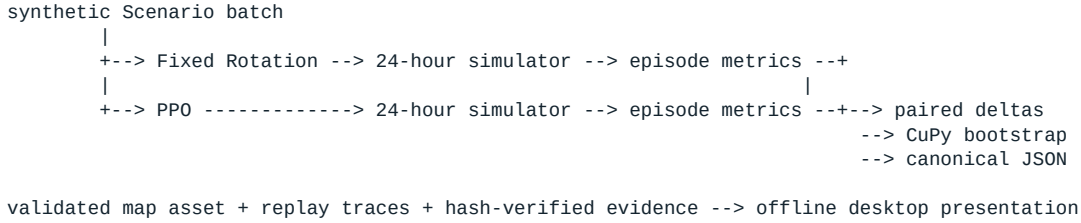
Limitations and threats to validity

- All demand and supply traces are synthetic and are not calibrated against utility records.
- The seven localities share one abstract supply pool; the model omits network topology, feeder limits, losses, voltage, frequency, protection, restoration, and operator constraints.
- The one-hour resolution cannot represent sub-hour switching or transient grid behavior.
- The study compares one frozen PPO actor with one Fixed Rotation rule. It does not establish superiority over every heuristic, optimization method, or learned controller.
- The fixed seeds and shortage construction limit external validity. Bootstrap intervals remain conditional on this scenario design.
- Power-balance compliance is an invariant of this simulator, not evidence of real-grid stability or operational safety.
- The 99% critical-outage threshold and reward weights are explicit research assumptions.
- The schematic map is a presentation aid, not a geographic boundary or electrical-network map.
- The causal projection has no real-data validation and is excluded from controller claims.

Implementation

System overview

GridGuard Khartoum is an offline Python 3.12 desktop application and reproducible CUDA simulation package. It separates four concerns: immutable domain contracts, deterministic scenario and controller execution, audited evidence generation, and presentation-only UI rendering.



The two controller paths share the exact same `Scenario` object. The desktop presentation may replay a selected scenario, but its Results view reads committed evidence rather than silently recalculating the headline claim.

Module map

Path	Responsibility
`src/gridguard/domain.py`	Locality, sector, shortage, service-tier, controller, scenario, result, and metric contracts
`src/gridguard/scenarios.py`	Seeded 24-hour synthetic demand and supply batches
`src/gridguard/controllers.py`	Shared controller protocol, Fixed Rotation rule, and PPO adapter
`src/gridguard/simulation.py`	Causal 32-value observation, tier dispatch, reward, and immutable result traces
`src/gridguard/metrics.py`	Per-episode proposal outcomes on the source tensor device
`src/gridguard/policy.py`	Strict PPO actor schema, artifact verification, CUDA load, and deterministic inference
`src/gridguard/statistics.py`	CuPy stratified paired bootstrap through DLPack
`src/gridguard/evaluation.py`	Exact scenario pairing, per-episode evidence, frozen parity, canonical serialization, and semantic verification
`src/gridguard/forecast.py`	Causal trailing-window demand projection for display only
`src/gridguard/gpu.py`	Fail-closed CUDA, PyTorch, CuPy, driver, and DLPack preflight
`src/gridguard/provenance.py`	Pinned source-manifest and artifact verification
`src/gridguard/resources.py`	Safe source-checkout or installed-bundle resource resolution
`src/gridguard/ui/map_asset.py`	Closed-schema validation for the offline locality schematic
`src/gridguard/ui/app.py`	Desktop entry point and fail-closed startup sequence
`src/gridguard/ui/window.py`	1366 x 768 shell, page navigation, and presentation shortcuts
`src/gridguard/ui/presentation_model.py`	Strict projection of paired evidence into presentation data
`src/gridguard/ui/map_view.py` and `src/gridguard/ui/map_items.py`	Interactive `QGraphicsView` map and redundant service-state rendering
`src/gridguard/ui/live_models.py`	Immutable, validated host-side contracts for two 24-hour controller timelines
`src/gridguard/ui/live_runner.py`	Shared-scenario CUDA execution and one explicit display transfer
`src/gridguard/ui/live_worker.py`	Qt worker signals that keep CUDA execution off the GUI thread
`src/gridguard/ui/overview_page.py`	Scope, protocol, and locality overview
`src/gridguard/ui/live_page.py`	Shortage/seed controls, 24-hour replay timeline, and locality detail surface
`src/gridguard/ui/results_page.py`	Artifact-derived controller comparison and paired intervals
`src/gridguard/ui/methodology_page.py`	In-application method and limitation summary
`src/gridguard/ui/components.py` and `src/gridguard/ui/theme.py`	Reusable accessible cards, metric formatting, and visual theme
`src/gridguard/cli.py`	`gridguard-doctor` and evidence-verification entry points

Domain and tensor contracts

The public vocabulary is encoded directly as enums. There are seven localities, five sectors, three shortage levels, and three service tiers. `Scenario` tensors have shapes:

```
demand: [batch, 24, 7, 5]
supply: [batch, 24]
```

Simulation traces have shapes:

```
service_tiers: [batch, 24, 7]
requested:     [batch, 24, 7, 5]
served:        [batch, 24, 7, 5]
reward:        [batch, 24]
```

Constructors reject invalid dimensions, mismatched devices or dtypes, negative power, non-finite values, invalid tiers, and inconsistent energy identities. Frozen dataclasses limit accidental reference reassignment, and the simulator does not mutate the scenario tensors.

GPU-first execution

The presentation and benchmark require `cuda:0`; there is no silent CPU fallback. PyTorch runs scenario generation, the 24-hour batched simulator, PPO inference, and metric calculation. CuPy runs the bootstrap sampling and percentiles. The large arrays remain on the GPU.

`run_gpu_preflight()` checks:

- PyTorch 2.8.0 with its CUDA 12.8 build;
- at least one visible NVIDIA device;
- CuPy in the supported range `>=13.6,<16` with a CUDA 12 runtime;
- an NVIDIA driver new enough for the CuPy runtime; and
- coherent zero-copy PyTorch/CuPy sharing by mutating a Torch CUDA tensor and observing the updated values through a CuPy DLPack view.

Any failed check stops launch or evaluation with an actionable error. Small unit tests can inject a CPU device for local logic checks, but the release gate runs the unfiltered GPU and source-provenance tests on the presentation laptop.

PPO artifact and inference

The desktop does not deserialize the source training checkpoint. A dedicated exporter extracts only the actor tensors into `data/model/ppo-policy.npz`. Runtime loading uses `numpy.load(..., allow_pickle=False)` and accepts exactly six float32 tensors with locked names and shapes.

Layer	Shape
First actor encoder weight	`[512, 32]`
First actor encoder bias	`[512]`
Second actor encoder weight	`[512, 512]`
Second actor encoder bias	`[512]`
Joint-action output weight	`[2187, 512]`
Joint-action output bias	`[2187]`

`data/model/manifest.json` binds the artifact to the accepted source checkpoint and source revision. Before loading CUDA parameters, the verifier checks the externally pinned SHA-256, file size, schema, architecture metadata, tensor set, dtype, shape, and finiteness. The loaded actor is moved to `cuda:0`, frozen, placed in evaluation mode, and restricted to deterministic argmax actions.

Scenario, dispatch, and metrics pipeline

`generate_scenarios()` creates a complete batch directly on the requested PyTorch device. A device-local seeded generator supplies both demand and supply perturbations. The simulator then iterates the 24 hours while keeping cumulative demand, cumulative served energy, and previous tiers on the same device.

At each hour:

1. Build the causal 32-value observation from current and accumulated information.

2. Ask the selected controller for seven integer tiers.
3. Reject malformed controller output before dispatch.
4. Convert tiers to requested sector loads.
5. Scale all requested loads proportionally if requests exceed available supply.
6. Update cumulative state and calculate the accepted reward.
7. Append immutable tensor traces for final metric calculation and presentation replay.

The metric module calculates one device-resident value per episode. Result and metric contracts, together with invariant tests, enforce `served + curtailed = demand`, verify served energy cannot exceed demand or availability, and report any hour exceeding the 0.001 MW power-balance tolerance.

Paired evaluation and statistics

`evaluate_scenario_pair()` executes Fixed Rotation and PPO on the exact same `Scenario` object. The full evaluator loops through three shortage levels and three seeds, generating a batch of 32 scenario variants for each of nine cells. The three shortage variants at a given `(seed, batch_index)` reuse one noise draw, so the design has 96 base-draw clusters. Crossing those draws with three shortage levels produces 288 matched scenario conditions, 288 paired rows, and 576 controller-scenario evaluations.

Episode metrics are stacked into `[3 shortage levels, 3 seeds, 32 episodes, 12 metrics]` cubes. On CUDA, the evaluator calculates `PPO - Fixed Rotation` in float64, selects the seven inferential metrics, and retains the `[3, 3, 32, 7]` dependence structure. For each of 10,000 deterministic replicates, CuPy samples 32 cluster indices with replacement inside each seed and reuses those indices across all three shortage variants. The statistic then gives equal weight to shortage levels, seeds, and sampled clusters. Endpoint calculation uses an explicit CUDA sort and linear interpolation, `a + (b - a) * weight`. This fixes the operation order and avoids reliance on the library percentile helper while preserving the defined linear percentile. Mean, lower, and upper values return to PyTorch in one stacked DLPack transfer. Only one packed numeric transfer returns the two metric cubes and bootstrap output to host memory. The canonical builder then materializes the 288 small row mappings and computes their cell/aggregate summaries in NumPy/Python for deterministic JSON serialization and validation; no controller, simulator, metric, or bootstrap computation falls back to CPU.

The resulting `data/evidence/paired-benchmark.json` contains:

- all 288 controller-paired episode rows;
- nine cell summaries;
- aggregate controller means;
- paired mean differences and 95% percentile intervals;
- metric definitions and direction metadata;
- frozen-cell parity errors and tolerances;
- exact scenario counts, seeds, and bootstrap protocol;
- the 96-base-draw cluster structure and three shortage variants per draw;
- PPO and source checkpoint provenance; and
- explicit scientific limitations.

Serialization is canonical UTF-8 JSON with sorted keys, finite numbers only, compact separators, and a final newline. The verifier requires the external hash, canonical byte representation, closed schema, exact row ordering, unique scenario identities, matched input energy, recomputed row deltas, metric physical bounds and energy identities, recomputed cell and aggregate means, inference metadata, and passing frozen parity. Verification also reloads the frozen artifact and cross-checks all 18 controller cell summaries rather than trusting the stored parity declaration alone.

Evidence provenance and fail-closed behavior

`data/evidence/source-manifest.json` is itself pinned in code. It contains exact IDs and hashes for the accepted proposal, frozen benchmark, PPO artifact, PPO manifest, paired benchmark, and nine source-lineage files. Verification rejects a rewritten manifest even if its internal entries are self-consistent.

Additional safeguards include:

- duplicate JSON fields and non-finite JSON constants are rejected;
- resource paths must remain relative to the runtime root;
- model files must match both an external trust anchor and their manifest;
- evidence cell counts, controller labels, seeds, and scenario IDs are closed rather than inferred from arbitrary input;
- the paired file must match SHA-256 `2c3e71e48c4c7bff4fe609d42a76ca4f770f6dd717cd7d45d5e8e0d99fdf2d11`; and
- the evaluator refuses devices other than `cuda:0`, bootstrap counts other than 10,000, or a bootstrap seed other than 20260815.

Display-only forecast

`causal_demand_projection()` accepts `[batch, hour, 7, 5]` demand tensors. Its default output is a one-hour-ahead trailing mean over the previous three hours. The function validates shape, dtype, finiteness, non-negativity, window, and horizon, preserves the input dtype and device, and uses `torch.inference_mode()`.

The live CUDA runner sums the sector projections into one value per locality and aligns them so current hours 2 through 22 display forecasts for hours 3 through 23. The first two hours lack a three-hour history and hour 23 has no next-hour target, so those frames display no projection. The module, runner, UI labels, and tests enforce the boundary: the projection is displayed only and is never passed to PPO, dispatch, reward, or the evidence generator.

Offline schematic map

The bundled map is `src/gridguard/assets/map/khartoum-schematic.json`. It contains only display geometry, locality labels, directional keyboard neighbors, and schematic Nile paths. Its own disclaimer states that it is neither a geographic boundary map nor an electrical-network topology.

The loader rejects duplicate fields, non-finite coordinates, unsupported schema values, unknown or missing localities, invalid keyboard order, external references, scientific values, controller values, degenerate or self-intersecting polygons, out-of-scene points, label anchors outside polygons, and invalid neighbor targets. Keeping simulation data out of the map asset prevents presentation geometry from becoming an unreviewed scientific input.

Desktop presentation

`gridguard-desktop` starts the PySide6 application through `ui/app.py`. Before it creates the main window, startup runs the complete project preflight: GPU and DLPack checks, pinned source manifest, paired-evidence semantics and frozen parity, PPO artifact verification, and a golden CUDA inference action. It then loads the same pinned paired-evidence snapshot and bundled map. A GPU, evidence, model, resource, or map failure blocks the presentation instead of opening a partly trusted interface.

The main window is designed at 1366 x 768 and contains four pages: Overview, Live Scenario, Results, and Methodology. Ctrl+1 through Ctrl+4 select the pages and F11 toggles fullscreen. Controls expose accessible names and descriptions, accept keyboard focus, and use a visible gold focus outline.

The `QGraphicsView` map supports pointer selection, hover details, arrow-key navigation through explicit locality neighbors, wheel or plus/minus zoom, drag-to-pan, and R or 0 to reset the view. Service state is never communicated by color alone: fill, texture, symbol, and text distinguish Off, Critical Only, and Full Service. The map retains its schematic disclaimer in the presentation.

The Live Scenario page accepts a shortage level and seed and starts a `QThread` worker so CUDA execution does not block the Qt event loop. The runner generates one `cuda:0` scenario, passes the exact same object through PPO and Fixed Rotation, and validates that every scenario and result tensor remains on `cuda:0`. Inputs are disabled while the worker is active, and the window refuses to close until the CUDA run has completed safely.

After both controllers finish, the runner packs hourly supply, 35 locality-sector demand values, seven next-hour locality projections, seven PPO tiers, 35 PPO served values, seven Fixed Rotation tiers, and 35 Fixed Rotation served values into one `[24, 127]` display tensor. This is the UI runner's only bulk device-to-host transfer. The resulting immutable comparison must contain exactly 24 frames for each named controller and identical supply, system demand, locality demand, and projection values at every paired hour.

A controller selector and 24-hour slider let the presenter compare tier allocations on the same scenario. The selected-locality panel derives actual demand MW, served MW, critical-service ratio, and service ratios for Hospital, Water, Bakery, Residential, and Commercial from the transferred simulation values. It also labels the causal next-hour demand projection as display only and explicitly states that the value is not PPO input or evidence. Until a valid comparison arrives, the page displays an honest waiting state; it never fabricates replay data. Runtime errors clear the frames, disable replay controls, and remain visible to the presenter.

The Results page renders only values present in the paired evidence snapshot, including paired intervals. It does not calculate replacement headline values. The Methodology page summarizes the locked design and repeats the evidence limitations for presentation use.

Runtime resources and packaging

`runtime_root()` supports two layouts:

1. a source checkout containing `data/evidence/source-manifest.json`; or
2. an installed package containing a `_bundled` runtime tree.

The wheel configuration force-includes `data/evidence`, `data/model`, and the accepted proposal PDF. The map JSON is package data under `src/gridguard/assets`. The same safe resolver is used for source and installed execution, and path traversal outside the chosen root is rejected.

The application is designed for an offline Windows presentation. No evidence, map geometry, model weight, or replay dependency requires a network request after installation.

Verification strategy

The tests are organized by the same boundaries as the implementation:

Test group	What it proves
Domain and scenario tests	Shapes, vocabulary, seed determinism, shortage construction, and GPU generation
Controller and simulation tests	Fixed schedule, action validation, dispatch, reward, and golden traces
Metric tests	Energy identities, proposal definitions, sign conventions, and finite device-resident outputs
Policy tests	Safe artifact parsing, actor architecture, source parity, decoding, and real CUDA inference
Evaluation and statistics tests	Exact pairing, counts, bootstrap stratification, DLPack sharing, canonical evidence, and frozen parity
Provenance and source tests	Pinned manifest, source Git blobs, accepted benchmark migration, and tamper rejection
Forecast tests	Causality, shape, dtype, device preservation, and invalid-input rejection
Map and package tests	Geometry, keyboard metadata, display-only schema, wheel inclusion, and isolated install loading

Test group	What it proves
Desktop UI tests	Startup-gate ordering, evidence projection, tabs, shortcuts, map interaction, and asynchronous live success/error states
GPU integration tests	Real PyTorch/CuPy/CUDA preflight on the presentation laptop

The portable command excludes release-only GPU and source-archive checks:

```
uv run pytest -m "not gpu and not source_archive"
```

The presentation release requires all checks without skips:

```
$sourceArchive = (Resolve-Path -LiteralPath "..\aipower_rl" -ErrorAction Stop).Path
$env:GRIDGUARD_SOURCE_REPOSITORY = $sourceArchive
$env:GRIDGUARD_SOURCE_CHECKPOINT = (
    Resolve-Path -LiteralPath (
        Join-Path $sourceArchive "artifacts\checkpoints\best.pt"
    ) -ErrorAction Stop
).Path
uv run gridguard-doctor
uv run pytest -ra
uv run gridguard-evaluate --verify-evidence
uv run ruff check .
```

Release review requires zero skipped tests. Setting both source paths explicitly prevents the source-parity fixtures from treating an absent research archive as an optional environment.

Deliberate implementation limits

- Training code and optimizer state are not part of the clean presentation runtime.
- The simulator is a decision-support abstraction and has no hardware control interface.
- The map contains no network topology or real administrative boundaries.
- NumPy/Python is restricted to artifact parsing and deterministic row/cell/aggregate assembly after the single final evidence transfer; simulation, inference, metrics, and bootstrap hot paths remain on the GPU.
- The result claim is fixed to the two named controllers, three shortage levels, three seeds, and one frozen PPO actor.

Results

Result statement

Under the locked synthetic evaluation design, PPO outperformed Fixed Rotation on all six primary outcomes: episode reward, critical-service ratio, utilization, Jain fairness, served energy, and curtailed energy. The newly reproduced per-episode artifact crosses 96 independent base draws with three shortage levels, producing 288 matched scenario conditions and 576 controller-scenario evaluations. For every primary outcome, the seed-stratified paired cluster 95% percentile interval lies wholly in the direction favoring PPO.

This is a simulation result, not a claim about real utility operation. It applies to one frozen PPO actor, one Fixed Rotation baseline, and the stated synthetic shortage design.

Two evidence layers

The project retains two complementary artifacts and does not treat them as interchangeable.

Evidence layer	File and SHA-256	Granularity	Permitted use
Accepted aggregate evidence	`data/evidence/frozen-benchmark.json` - `3d720ba87a747a5780119decd7144b4051fc36fc72ab50f229f5750ad749fd15`	18 controller batch-cell summaries: 2 controllers x 3 shortage levels x 3 seeds; each summary represents 32 episodes	Preserve and reproduce the accepted aggregate comparison; no confidence intervals, p-values, or episode win rates
Reproduced paired evidence	`data/evidence/paired-benchmark.json` - `2c3e71e48c4c7bff4fe609d42a76ca4f770f6dd717cd7d45d5e8e0d99fdf2d11`	288 scenario-condition rows from 96 base-draw clusters, each containing both controller outcomes and their paired difference, plus 9 cell summaries	Aggregate results, matched differences, deterministic seed-stratified paired cluster intervals, and critical-outage locality-hours

The accepted artifact was migrated without inventing episode records. The new artifact was then regenerated from the pinned simulator and PPO actor, retained every episode pair, and passed a cell-level parity comparison against the accepted summaries.

Accepted aggregate comparison

The following values are the exact numeric values stored in the accepted aggregate artifact. They are means per 24-hour synthetic episode, not sums across 288 episodes. Ratios are shown as stored; multiply by 100 only when presenting them as percentages.

Metric	Fixed Rotation	PPO	Direction / role
Episode reward	13.969324111938477	23.512135399712456	Higher / primary
Critical-service ratio	0.4262974626488156	0.9971638056966994	Higher / primary
Utilization	0.7122341394424438	0.9765213396814134	Higher / primary
Jain fairness	0.8775761524836222	0.9691991872257657	Higher / primary
Served energy (MWh)	4551.691569010417	6560.189181857639	Higher / primary
Curtailed energy (MWh)	6142.390950520833	4133.893283420139	Lower / primary
Switching rate	0.2500000298023224	0.21887401905324724	Lower / secondary
Oversubscription	0.005967141646477912	0.0017833372629765007	Lower / diagnostic
Power-balance violation rate	0	0	Zero / safety
Demand energy (MWh)	10694.082356770834	10694.082356770834	Matched input
Available energy (MWh)	6710.389431423611	6710.389431423611	Matched input

The source stored only these cell-derived aggregates, so inferential statistics are intentionally absent from this evidence layer.

Reproduced paired aggregate comparison

The following are the exact aggregate values stored in the new per-episode paired artifact. They are means per 24-hour synthetic episode. Small differences from the preceding table reflect reproduced float32 reduction order and, for critical service, episode-ratio averaging rather than pooled energy. All are within the locked parity tolerances shown below.

Metric	Fixed Rotation	PPO	Direction / role
Episode reward	13.969324383470747	23.512135558658176	Higher / primary
Critical-service ratio	0.4262975813407037	0.997163764304585	Higher / primary
Utilization	0.712234134061469	0.976521334093478	Higher / primary
Jain fairness	0.8775761361337371	0.9691991855700811	Higher / primary
Served energy (MWh)	4551.691485934787	6560.189144558377	Higher / primary
Curtailed energy (MWh)	6142.390977647569	4133.893338521321	Lower / primary
Switching rate	0.2500000298023224	0.21887402122633326	Lower / secondary
Oversubscription	0.005967141698218054	0.0017833372334684504	Lower / diagnostic
Power-balance violation rate	0	0	Zero / safety
Critical outage locality-hours	102.07291666666667	18.666666666666668	Lower / diagnostic
Demand energy (MWh)	10694.08246866862	10694.08246866862	Matched input
Available energy (MWh)	6710.389377170139	6710.389377170139	Matched input

The aggregate critical-service ratio corresponds to approximately 42.63% for Fixed Rotation and 99.72% for PPO. PPO also reduced mean critical outage exposure from approximately 102.07 to 18.67 locality-hours per episode. Critical outage is a diagnostic threshold measure and was not included in the bootstrap interval set.

Both controllers recorded an aggregate power-balance violation rate of exactly zero in the paired artifact. This confirms the implemented simulator invariant for these runs; it is not a real-grid stability claim.

Paired differences and 95% intervals

Each difference below is exactly `PPO - Fixed Rotation`. The table reproduces the stored mean differences and deterministic 95% stratified paired percentile intervals. Values are copied from the pinned paired artifact without re-rounding.

Metric	Mean difference	95% lower	95% upper	Favorable sign
Episode reward	9.54281117518743	9.53561696095599	9.550073716955053	Positive
Critical-service ratio	0.5708661829638811	0.570613363581813	0.5711048985391648	Positive
Utilization	0.26428720003200906	0.2632496295703782	0.2652864054621508	Positive
Jain fairness	0.09162304943634403	0.09091882564437886	0.09230119484062825	Positive
Served energy (MWh)	2008.4976586235891	2000.3781833224828	2016.7392950693768	Positive
Curtailed energy (MWh)	-2008.4976391262478	-2016.7392876095244	-2000.3781435224746	Negative
Switching rate	-0.031126008575989142	-0.03478424262462391	-0.0274884424974314	Negative

All seven intervals exclude zero in the favorable direction. This statement is descriptive of the locked, seed-stratified cluster bootstrap distribution; no p-value or multiplicity-adjusted hypothesis test was calculated. In presentation terms, the ratio differences correspond to about +57.09 percentage points in critical service, +26.43 percentage points in utilization,

+9.16 percentage points in Jain fairness, and -3.11 percentage points in switching.

Served and curtailed energy are complementary under the energy identity. Their paired results describe the same allocation change with opposite signs and should not be interpreted as two independent endpoints.

Consistency across the nine evaluation cells

Each shortage-level and seed combination is a separate 32-episode cell. PPO had the favorable cell mean in all 9 of 9 cells for every primary outcome. The observed cell-mean difference ranges were:

Primary outcome	Favorable cells	Minimum cell difference	Maximum cell difference
Episode reward	9 / 9	8.649745643138885	10.428759038448334
Critical-service ratio	9 / 9	0.5634181443601847	0.5836229948326945
Utilization	9 / 9	0.05924975872039795	0.43822693079710007
Jain fairness	9 / 9	0.082805709913373	0.09917639568448067
Served energy (MWh)	9 / 9	296.61602783203125	3757.100311279297
Curtailed energy (MWh)	9 / 9	-3757.100296020508	-296.6159973144531

The secondary switching-rate cell mean favored PPO in 7 of 9 cells even though its overall paired interval favored PPO. Oversubscription was lower in aggregate for PPO but its direction was mixed across shortage levels and cells: PPO was lower in 3 cells and higher in 6. For that reason, oversubscription remains diagnostic and is not presented as a consistent primary advantage. Power-balance violations were zero in every cell for both controllers.

Parity with the accepted aggregate evidence

Before the per-episode results were serialized, every reproduced controller cell summary was compared with its accepted counterpart. The comparison covered 18 summaries and used fixed, code-locked absolute release tolerances. All metrics passed.

Metric	Maximum absolute cell error	Allowed absolute tolerance
Available energy (MWh)	0.00048828125	0.002
Critical-service ratio	0.0000012684613466262817	0.000002
Curtailed energy (MWh)	0.000274658203125	0.002
Demand energy (MWh)	0.00091552734375	0.002
Episode reward	0.0000011920928955078125	0.000002
Jain fairness	8.754432201385498e-8	0.000002
Oversubscription	8.149072527885437e-10	0.000002
Power-balance violation rate	0	0.000002
Served energy (MWh)	0.0006103515625	0.002
Switching rate	8.847564458847046e-9	0.000002
Utilization	8.754432201385498e-8	0.000002

Critical outage locality-hours is absent from the accepted aggregate artifact, so it has no frozen parity row. It is calculated from the newly retained per-episode traces using the locked 99% service threshold.

For critical-service ratio specifically, the accepted source cells used pooled critical energy, whereas the paired artifact calculates episode ratios before averaging. The recorded maximum cell difference of `0.0000012684613466262817` remains below the locked tolerance. The paired rows are therefore a compatible reproduction, not reconstructed historical episode records.

Interpretation against the proposal questions

- **Critical services:** PPO delivered nearly all modeled critical-sector demand and sharply reduced critical outage locality-hours relative to Fixed Rotation.
- **Fairness:** PPO increased the mean hourly Jain fairness of cumulative locality service ratios.
- **Distribution efficiency:** PPO increased utilization and served energy while reducing curtailed energy.
- **Operational consistency inside the model:** both controllers respected the simulator's supply constraint, and PPO had a lower aggregate switching rate.
- **Traditional schedule comparison:** the matched design isolates controller behavior from scenario differences because both controllers receive identical demand and supply tensors.

These findings satisfy the proposal-level software demonstration and comparison objectives. They do not establish national-grid optimization, field readiness, or performance under real utility data.

Limitations of the result

- The scenarios are synthetic, use fixed demand shares and hourly profiles, and contain no measurements from a utility.
- The simulator uses one abstract supply pool and omits electrical-network constraints and sub-hour dynamics.
- The paired intervals resample 96 base-draw clusters inside three fixed seed strata while keeping each cluster's three shortage variants together. They do not describe uncertainty across all possible seeds, shortage prevalence, or future grid conditions.
- Only PPO and Fixed Rotation are included in the confirmatory comparison.
- Results describe one frozen PPO actor under deterministic argmax inference.
- The reward weights and 99% critical-outage threshold are modeling choices.
- The display-only demand projection is not part of this evidence and has no reported accuracy.
- The offline locality map is schematic and contributes no scientific input to the evaluation.

Conclusion

Under the locked synthetic evaluation protocol, PPO is consistently better than Fixed Rotation on every primary outcome and all nine shortage-by-seed cells. The clean release makes that result reproducible, preserves the accepted aggregate evidence, and presents the method honestly through an offline GPU-first desktop application. The next scientific step is validation with representative operational data and constraints before any field-facing use.

Canonical paired evidence SHA-256: 2c3e71e48c4c7bff4fe609d42a76ca4f770f6dd717cd7d45d5e8e0d99fdf2d11

References

1 GridGuard Khartoum project team. *Graduation Project Proposal: AI-Based Smart Load Shedding and Energy Management System for Developing Countries*. Unpublished project proposal, archived at `docs/source/graduation-project-proposal.pdf`.

2 Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. "Proximal Policy Optimization Algorithms." *arXiv preprint arXiv:1707.06347*, 2017. DOI: 10.48550/arXiv.1707.06347.

3 Jain, R. K., Chiu, D.-M. W., and Hawe, W. R. "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems." DEC Research Report TR-301, 1984.

4 Efron, B., and Tibshirani, R. J. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993. DOI: 10.1201/9780429246593.

5 Paszke, A., Gross, S., Massa, F., et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library." *Advances in Neural Information Processing Systems 32*, 2019.

6 Okuta, R., Unno, Y., Nishino, D., Hido, S., and Loomis, C. "CuPy: A NumPy-Compatible Library for NVIDIA GPU Calculations." *Proceedings of the Workshop on Machine Learning Systems at NIPS 2017*, 2017.